

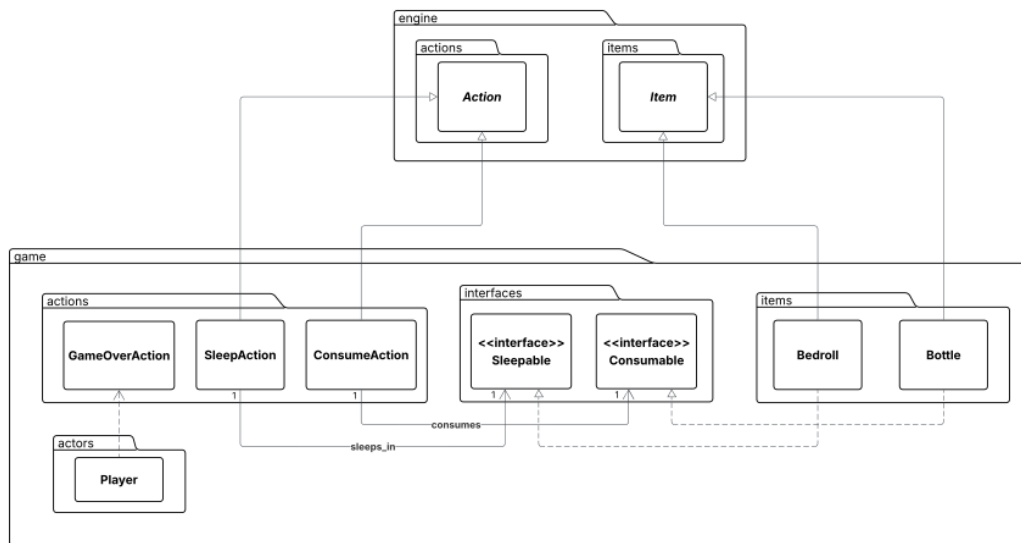
FIT2099 assignment 1

Shengyuan Jin

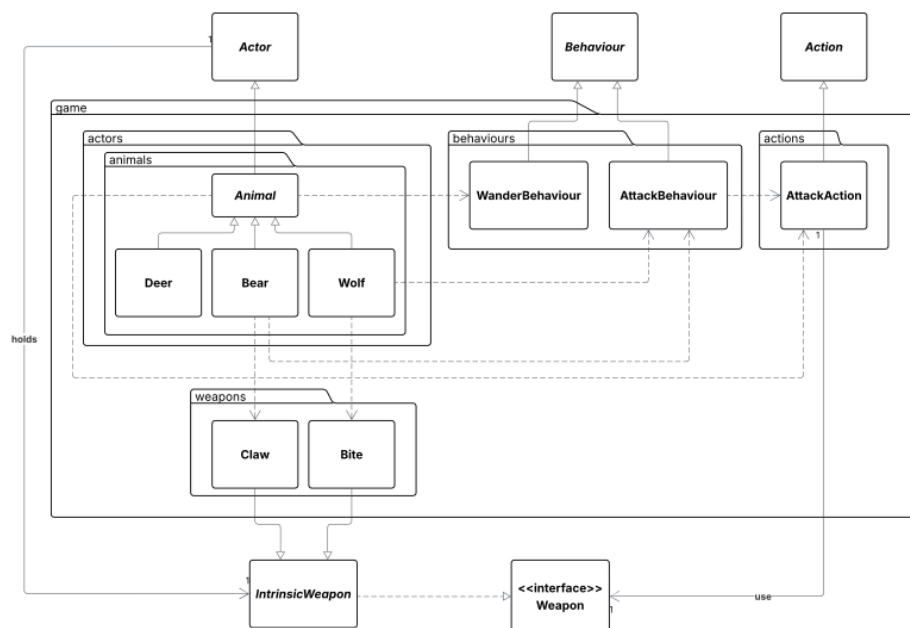
34417257

Design Diagrams

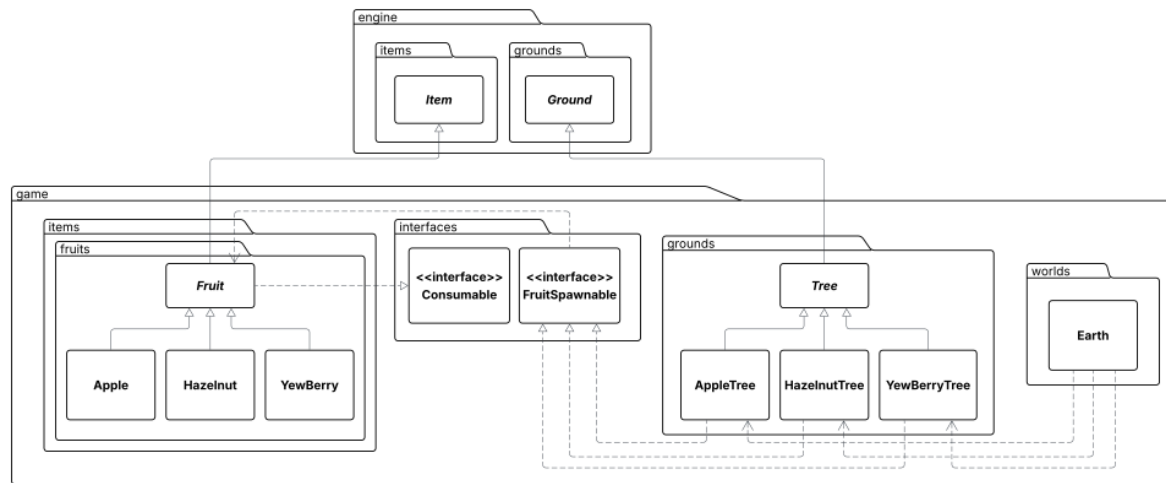
REQ 1



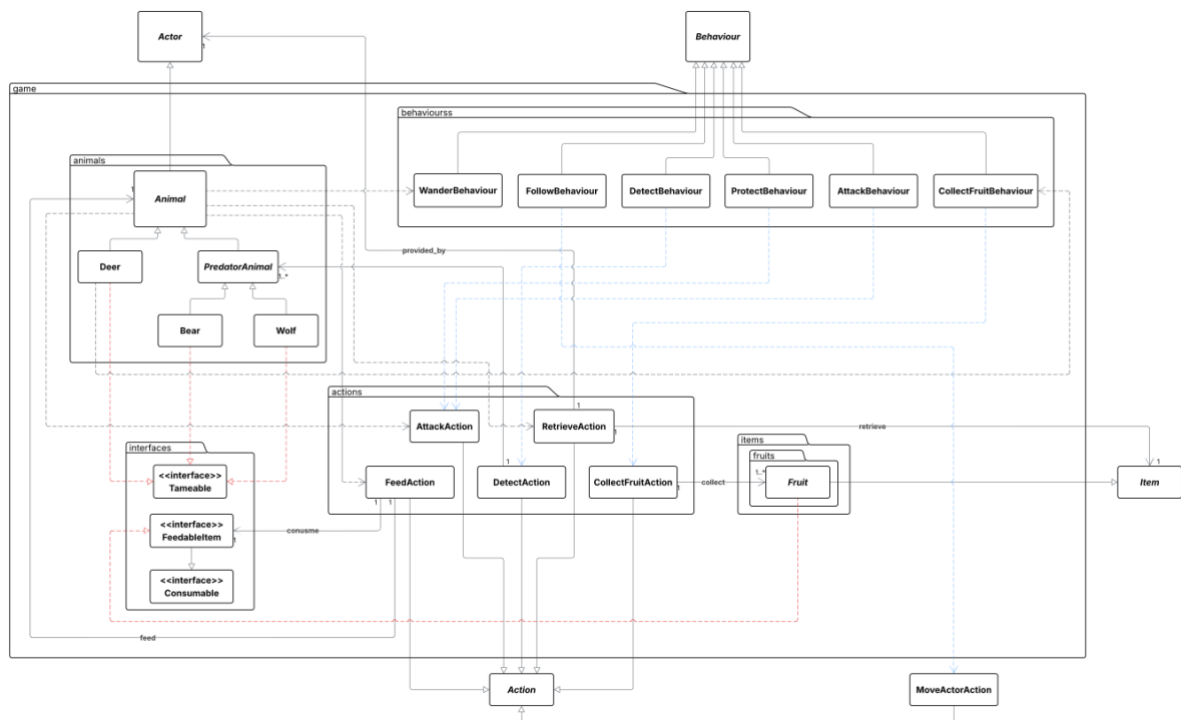
REQ 2



REQ 3



REQ 4



Design Rationale

REQ1

Design:

Create concrete class Bottle and Bedroll extending abstract class Item.

Create Sleepable interface and SleepAction class handle the sleeping functionality. Any object that can be slept on, such as a Bedroll, implements Sleepable. The SleepAction is then responsible for executing the sleeping behaviour implemented by Sleepable.

Similarly, the Consumable interface and ConsumeAction class manage consume behaviour. Objects like the Bottle implement Consumable and then implement its own effect in consumedBy(), which would then be triggered by ConsumeAction.

Additionally, I introduced the GameOverAction to represent the end of the game when survival thresholds are reached, which also extends Action. If either hydration or warmth reaches zero, the player becomes unconscious, and the GameOverAction is triggered with a context-specific message (e.g., “died of dehydration” or “froze to death”).

Advantages of the Design

Single Responsibility Principle (SRP)

Each class has a distinct responsibility:

- Bedroll and Bottle manage their own state (uses remaining/counter, etc.).
- SleepAction and ConsumeAction only handle the logic of performing the relevant action. This improves cohesion and makes the code easier to maintain.

Dependency Inversion Principle (DIP)

Actions (SleepAction, ConsumeAction) depend on the abstractions Sleepable and Consumable, rather than concrete classes. This makes the system flexible: introducing or modifying a new sleepable or consumable item would never affect the existing codebase.

Open-Closed Principle (OCP)

The design is open to extension but closed to modification. To add new items (e.g., a Tent for sleeping or a River for drinking), only a new class implementing the relevant interface is needed.

Low Coupling and High Cohesion

Each class encapsulates its own logic, while GameOverAction ensures that the game-over state is decoupled from the core player loop, keeping responsibilities distinct.

Disadvantages of the Design

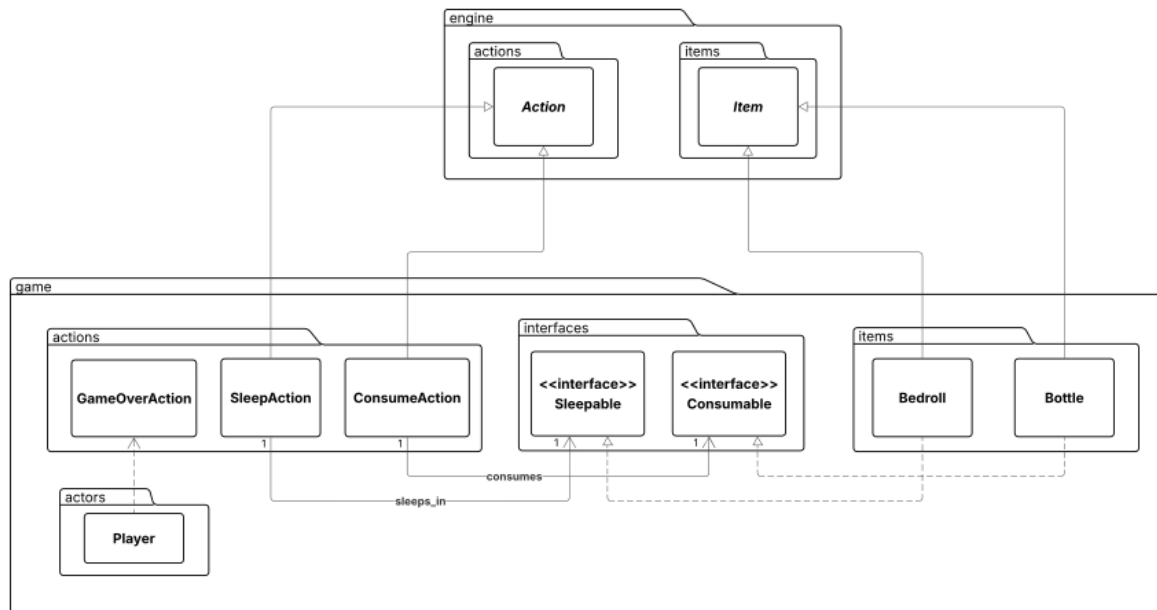
Scattered Logic (potential DRY issue)

The sleeping and consuming effects are defined in each item class (e.g., Bedroll, Bottle). If many similar items exist, this could result in duplicated logic across classes. For instance, same logic in allowableActions() or same operation on counter.

Limited Extensibility for Contextual Effects

Currently, the effects of sleeping or consuming are fixed in the item. If effects need to vary by context (e.g., sleeping during a storm restores less warmth), concrete item classes would need modification, slightly reducing adherence to OCP.

Conclusion



The diagram above shows the final design of requirement 1. Both new Bottle and Bedroll created extend the abstract Item class, ensuring shared attributes and methods are abstracted to avoid duplication (**DRY Principle**). Items rely on the constructor of the superclass to initialize their values, allowing them to be safely substituted in code (**Liskov Substitution Principle**).

By introducing Sleepable and Consumable interfaces, SleepAction and ConsumeAction depend on abstractions rather than concrete classes, ensuring that high-level modules are not directly tied to low-level details (**Dependency Inversion Principle**).

The addition of GameOverAction provides a modular and reusable way to represent terminal states, keeping the game loop clean and ensuring consistency when hydration or warmth reaches zero.

Overall, Design 1 provides cohesion, scalability, and maintainability, making it the most suitable choice for supporting future expansion of the survival game.

REQ2

Design 1

In this approach, weapons are paired with their own custom attack actions and behaviours. For instance, a **BiteAction** triggered by a **BiteBehaviour** encapsulates all the details of performing a bite attack (damage, hit rate, etc.), while a **ClawAction** does the same for claw attacks. These action classes can then be reused by any animal capable of performing that type of attack, offering some level of sharing.

Advantages of the Design

Single Responsibility Principle (SRP)

Each action (BiteAction, ClawAction) clearly encapsulates the logic of a specific type of attack, while the animal class itself only decides when to attack. This separation avoids overloading animal classes with both behaviour and combat execution.

Open-Closed Principle (OCP)

The design supports extension by allowing new attacks to be defined as new Action classes. For example, adding a ChargeAction or SwipeAction only needs to implement their corresponding Behaviour.

Disadvantages of the Design

Code Duplication

Even though weapon-specific actions can be reused across animals, much of their logic (accuracy checks, unconscious handling, menu description) will be almost identical. This introduces repeated code and increases the chance of inconsistencies, violating the DRY principle.

Reduced Flexibility

By binding attack logic to custom actions, it becomes harder to support new weapon effects without proliferating more action classes. For example, introducing a new weapon like “Charge” would require creating both ChargeAction and potentially a new ChargeBehaviour, instead of simply reusing the generic AttackAction with a new Weapon implementation.

Increased Maintenance Overhead

If the rules for attacking change globally (e.g., adding critical hits, environmental modifiers, or unconscious handling updates), every custom Action class must be updated.

Design 2

In this requirement, animals such as Deer, Bear, and Wolf inherit from the abstract Animal class, which itself extends Actor from the engine.

Animal decision-making is handled by behaviour classes such as WanderBehaviour and AttackBehaviour, both of which implement the Behaviour interface. The priority of these behaviours determines when an animal should move randomly or attack a nearby target.

Combat execution is encapsulated in the AttackAction class, which is triggered by behaviours rather than directly controlled by the animals. This ensures that combat logic is centralized and reusable.

Each hostile animal is equipped with intrinsic weapons, such as Claw or Bite, which extend the IntrinsicWeapon abstract class and implement the Weapon interface. The intrinsic weapons only need to define their damage, verb, hit rate, and name, while reusing the generic attack logic from the higher-level Weapon interface.

Advantage of the design

Single Responsibility Principle (SRP)

- Deer, Bear, and Wolf manage only their species-specific attributes.
 - WanderBehaviour and AttackBehaviour determine decision-making logic.
 - AttackAction only handles the mechanics of executing attacks.
 - Claw and Bite define the details of intrinsic weapons.
- This separation of responsibilities improves cohesion and makes debugging or extension straightforward.

Dependency Inversion Principle (DIP)

Behaviours (AttackBehaviour, WanderBehaviour) depend on the abstraction Behaviour, and attacks rely on the Weapon interface. This ensures neither behaviours nor actions depend on specific animal implementations, increasing flexibility.

Don't Repeat Yourself (DRY)

Intrinsic weapons such as Claw and Bite only need to specify their parameters (damage, hit rate, name, etc.). All intrinsic weapons reuse the generic attack() method provided by the higher-level Weapon abstraction. This eliminates duplicated combat logic and ensures all animals can share the same AttackAction.

Open-Closed Principle (OCP)

The design is open for extension. New animals (e.g., a Tiger) can be introduced by subclassing Animal and assigning intrinsic weapons. Similarly, new behaviours (e.g., FleeBehaviour) can be added without modifying existing code.

Low Coupling and High Cohesion

Animals, behaviours, and weapons are clearly decoupled. For example, a Wolf does not control how its attack is executed; it simply holds a Bite weapon, and the AttackAction performs the attack. Each class encapsulates its own logic, maintaining cohesion.

Disadvantage of the design

Limited Extensibility for Contextual Effects

Attack outcomes are fixed within intrinsic weapon classes. If outcomes need to vary by context (e.g., a Wolf's Bite deals more damage with some buff), the weapon class would need to be modified directly, slightly reducing adherence to the Open-Closed Principle.

Conclusion

AppleTree, an Apple object is injected into the FruitTree, and the tree uses the injected fruit for spawning.

Advantages of the Design

Reduced Code Duplication

Because spawnFruit() does not need to be overridden, all tree types can reuse the same spawning method. The only difference lies in the fruit object provided during construction.

Centralized Logic

All spawning behaviour is handled in FruitTree, ensuring consistency. If spawning rules change (e.g., adding probability, growth timers), the modification is only needed in one place.

Disadvantages of the Design

Reduced Flexibility

This design assumes a one-to-one mapping between tree and fruit. If a future tree can spawn multiple fruit types (e.g., seasonal fruits, rare drops), the injected single fruit attribute becomes too restrictive, and spawnFruit() would need rework.

Semantic Limitations

Non-fruit plants (e.g., decorative trees, poisonous trees that never spawn items) do not fit well into this inheritance hierarchy. They would either need to inject a “null fruit” or avoid extending FruitTree, which breaks consistency.

Harder to Vary Behaviour Dynamically

With injection, the fruit is fixed when the tree is created. If the design later requires trees to spawn different fruits depending on conditions (e.g., an AppleTree that sometimes spawns RottenApple), this pattern becomes harder to adapt.

Design 2

In this design, I create an abstract Tree class as the base for all different trees. However, because not all trees can spawn fruits, I just let each tree implements the FruitSpawnable interface, which allows it to generate a fruit item at defined intervals.

The fruit items (Apple, Hazelnut, YewBerry) extend the abstract Fruit class and implement the Consumable interface. This ensures that each fruit encapsulates its own consumption logic.

By separating Tree (spawning) and Fruit (consumption), the design ensures clear responsibilities and supports future extension for other plant types that may or may not produce items.

Advantages of the Design

Single Responsibility Principle (SRP)

- Trees only manage fruit spawning logic and timing.
 - Fruit and its subclasses only handle their consumption effects.
- This separation ensures high cohesion and easier testing of each component.

Dependency Inversion Principle (DIP)

The spawning system depends on the abstraction `FruitSpawnable`, not on concrete trees. Likewise, consumable actions depend on the abstraction `Consumable`, not on specific fruit types. This decoupling makes the design flexible and reusable.

Open-Closed Principle (OCP)

The system is open for extension but closed for modification. To add a new type of tree or fruit, only a new class implementing `FruitSpawnable` or `Consumable` is required. Existing logic in the Explorer, actions, or trees does not need modification.

Low Coupling and High Cohesion

Spawning and consumption are decoupled: trees produce items without knowledge of how they will be consumed, while fruits provide effects without depending on how they were created. Cohesion is preserved as each class focuses on a single domain responsibility.

Disadvantages of the Design

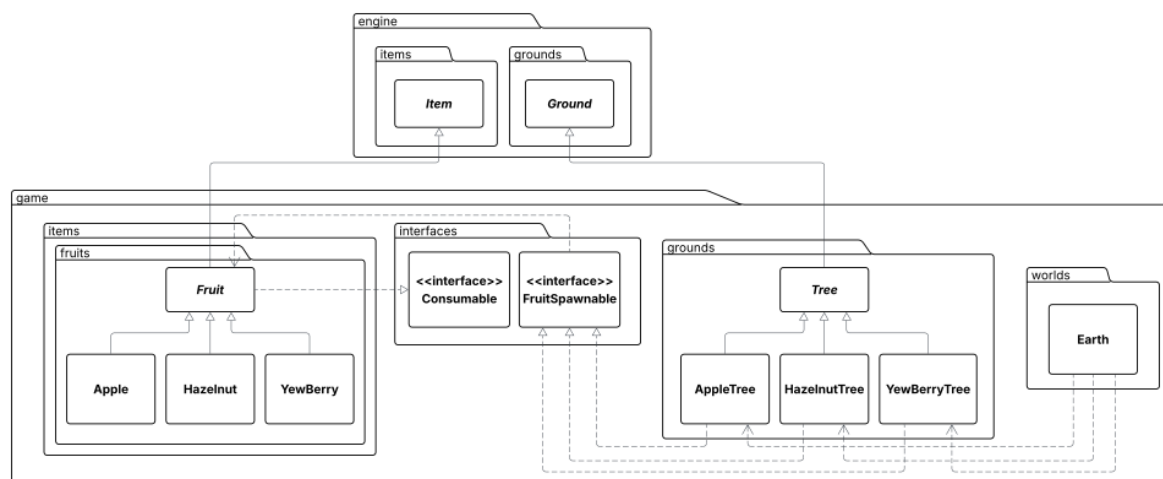
Scattered Logic Across Fruits

The consumption logic is defined separately in each fruit class (Apple, Hazelnut, YewBerry). If many fruits are added, similar logic (e.g., healing, hydration adjustment) could be repeated.

Limited Context Awareness

Currently, consumption effects are fixed per fruit type. If effects need to vary by context (e.g., eating a fruit during winter restores more hydration), the concrete fruit classes would need to be modified, slightly reducing adherence to OCP.

Conclusion



The diagram above shows the final design for requirement 3, which implements design 2. Each tree subclass extends the abstract Tree class, ensuring shared attributes and behaviours are abstracted to reduce duplication (DRY Principle).

Fruits extend the abstract Fruit class and implement the Consumable interface, encapsulating their consumption logic. By introducing the FruitSpawnable interface, trees depend on abstractions rather than concrete fruits, ensuring that high-level modules are not directly tied to low-level details (Dependency Inversion Principle).

Overall, this design provides clarity, scalability, and maintainability, making it suitable for supporting future expansion of flora, including trees that may or may not produce consumable items.

Req4

Part A: PredatorAnimal and Taming System

Design 1

we could define two abstract classes:

- **TameableAnimal** extends Animal and provides built-in support for taming. This abstract class would contain a Stance attribute, as well as methods such as tame() and isTamed(). Animals like **Bear**, **Wolf**, or **Deer** would extend TameableAnimal directly if they are designed to be tameable. Non-tameable animals would continue extending the regular Animal class.
- **FeedableItem** extends Item and still implements Consumable. Subclasses like Apple, Hazelnut, or YewBerry could extend this abstract class.

With this structure, the taming system becomes more rigidly integrated into the class hierarchy, avoiding the need for animals or items to implement multiple interfaces.

Advantages

- **Centralized Logic:** Shared taming logic in TameableAnimal and shared feeding logic in FeedableItem ensure that common functionality is centralized, reducing repeated checks.
- **Simplicity for Developers:** Developers only need to extend the appropriate abstract class instead of juggling multiple interfaces. This makes the design easier to understand.
- **Built-in Support:** Because TameableAnimal and FeedableItem already provide taming/feeding methods, subclasses need to implement fewer details themselves.

Disadvantages

- **Reduced Flexibility:** Using abstract classes ties subclasses to a fixed hierarchy. For example, an animal cannot be both a special subclass of Animal and also a TameableAnimal if it already extends another abstract class (Java does not allow multiple inheritance).
- **Breaks Separation of Concerns:** TameableAnimal mixes animal logic with taming logic, and FeedableItem mixes item logic with feeding logic. This reduces modularity compared to using interfaces.
- **Less Extensible in Complex Systems:** If later plants, potions, or other non-Item classes should be feedable, they cannot reuse FeedableItem because of the rigid inheritance.

Design 2

To clearly represent the state of an animal, I introduced the Stance enumeration with two values: HOSTILE and TAMED. By default, predators start in the HOSTILE stance. When they are successfully tamed, their stance is updated to TAMED, and if they were hostile, the HOSTILE stance is removed. I assumed that all animals can be **fed**, but not every animal can necessarily be **tamed**. To reflect this, I introduced two new interfaces:

- **Tameable** – implemented only by animals that can be tamed.
- **FeedableItem** – implemented by items that can be fed to animals.

To simplify the design, FeedableItem extends from Consumable. This allows reuse of existing consumption logic — for example, when an animal is fed a fruit, it also receives the same healing or effects as if the Explorer consumed it.

The FeedAction was introduced to handle the feeding process. A FeedAction becomes available in an animal's allowableActions only if the player has a FeedableItem in their inventory and the animal is nearby. This ensures that feeding opportunities are dynamically controlled by both context and inventory.

Advantages

- **Separation of Concerns:** Feeding and taming logic are separated from animal classes, keeping animals focused on state while actions manage execution.
- **Reuse of Existing Logic:** By letting FeedableItem extend Consumable, feeding automatically applies the same effects (e.g., healing, hydration) as consuming, making the system more consistent and logical.
- **Flexible Extensibility:** New tameable animals can be introduced simply by implementing Tameable, while new feedable items can be added by implementing FeedableItem. Additional stances (e.g., NEUTRAL, FRIENDLY) could be introduced in the future to represent more nuanced animal behaviours without redesigning the hierarchy. No existing code needs modification (OCP).
- **Contextual AllowableActions:** Feeding is only available when the right conditions are met (item + proximity), ensuring realism and preventing invalid actions.
- **Clear State Representation:** Stance provides an explicit and readable way to differentiate hostile vs. tamed animals, improving code clarity.

Disadvantages

- **Increased Interface Count:** Adding both Tameable and FeedableItem introduces more abstraction layers, which could make the design harder for new developers to follow.
- **Assumption of Shared Effects:** Reusing Consumable logic means animals and the Explorer experience identical effects when consuming the same item, which requires to add some if to check whether the consumer has such statistic, making code very complicated and tedious if there are many different statistics.
- **Extra Checks Required:** Each tameable animal's allowableActions() must include stance checks, introducing some redundancy across animals. As more stances are added, the logic in allowableActions() could become more complex and harder to maintain.

Part B: Post-Taming Behaviours

Design

Once an animal is successfully tamed, its behaviour set changes to reflect its new role alongside the Explorer. All tameable animals gain **FollowBehaviour**, ensuring they stay close to the player.

Additionally, different animals gain unique behaviours based on their type:

- **Deer:** Gains CollectFruitBehaviour, allowing them to pick up dropped fruits. To return collected fruits, I introduced a new RetrieveAction, which becomes available only when the tamed Deer is adjacent to the player.
- **Bear:** Gains ProtectBehaviour, making it fight against hostile predators to defend the Explorer.
- **Wolf:** Gains both ProtectBehaviour and DetectBehaviour, making it capable of both defending and alerting against hostile threats.

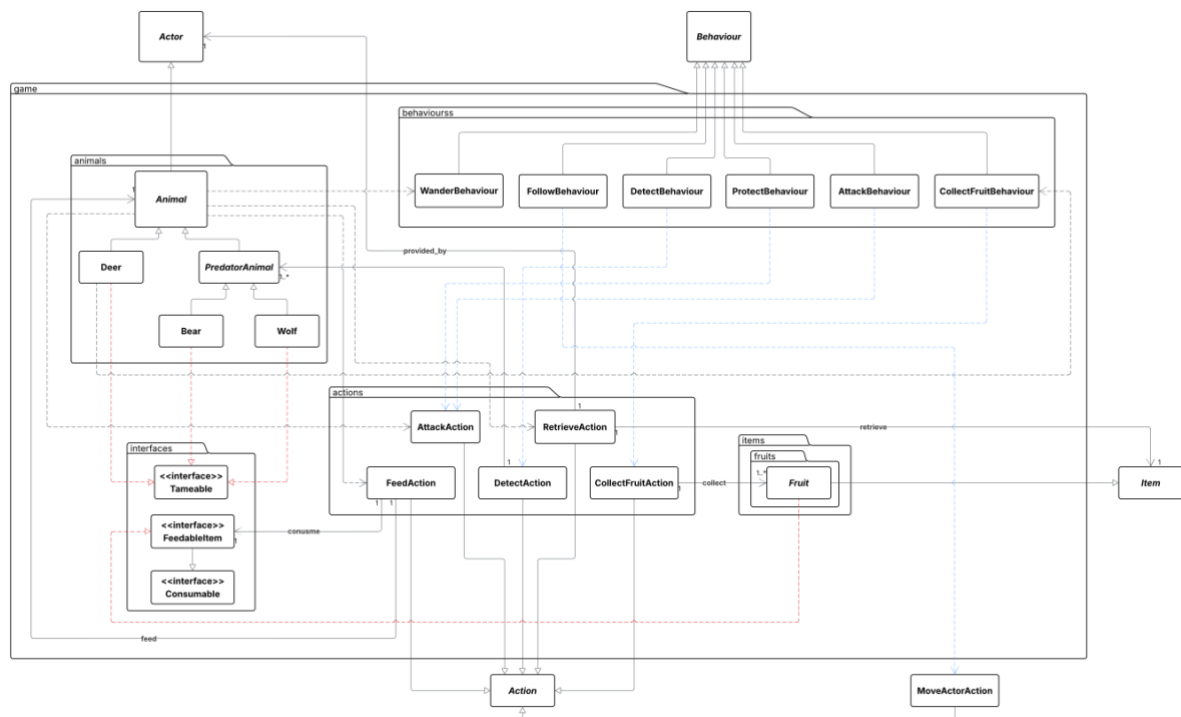
This separation ensures that taming does not make all animals behave identically but instead gives them unique and meaningful roles.

Advantages

- **Rich Gameplay Variety:** Different tamed animals provide distinct benefits (combat defence, detection, fruit collection), making taming a strategic choice.
- **Polymorphic Behaviours:** Reusing the Behaviour abstraction allows animals to dynamically gain or lose behaviours at runtime, without modifying their class definitions.
- **Encapsulation of Actions:** RetrieveAction encapsulates the logic of taking items from helper animals, without cluttering the Deer class itself.

Disadvantages

- ## Conclusion



To distinguish the state of animals, the Stance enumeration was introduced, where predators start as HOSTILE and change to TAMED when successfully tamed. This stance directly controls which actions appear in an animal's allowableActions, ensuring that available interactions always reflect the animal's role toward the player.

Overall, the design balances clarity, modularity, and extensibility, providing a robust framework for dynamic animal behaviours in both hostile and allied contexts.